ForecastAI

Version 2026.01

Technical Documentation & User Guide

Enterprise Demand Forecasting Platform

Multi-Method Statistical & ML Forecasting with Automated Model Selection

Generated: February 2026

Table of Contents

1. **Executive Summary**
2. **System Architecture**
   - 2.1 Technology Stack
   - 2.2 Pipeline Architecture
   - 2.3 Directory Structure
3. **Data Model**
   - 3.1 Source Database
   - 3.2 Output Parquet Files
4. **Pipeline Stages**
   - 4.1 ETL (Extract-Transform-Load)
   - 4.2 Time Series Characterization
   - 4.3 Forecasting
   - 4.4 Backtesting & Evaluation
   - 4.5 Best Method Selection
   - 4.6 Distribution Fitting (MEIO)
5. **API Reference**
6. **Frontend User Guide**
   - 6.1 Dashboard
   - 6.2 Time Series Detail View
   - 6.3 Interactive Features
7. **Configuration Reference**
8. **Installation & Running**
9. **Troubleshooting**

1. Executive Summary

ForecastAI 2026.01 is an enterprise-grade demand forecasting platform that automates the end-to-end process of time series forecasting. It extracts demand data from a Neon PostgreSQL database, characterizes each time series for patterns such as seasonality, trend, and intermittency, generates forecasts using up to 15 methods across 4 model families, evaluates forecast accuracy via rolling-window backtesting, selects the best method per series, and fits probability distributions for Multi-Echelon Inventory Optimization (MEIO).

The system serves its results through a FastAPI REST backend and a React/Vite interactive dashboard, enabling users to explore forecasts, compare methods, zoom into specific time periods, and review accuracy metrics across all 2,348 time series in the dataset.

Key Capabilities

- Automated ETL from PostgreSQL via DuckDB with monthly aggregation
- Time series pattern detection: seasonality, trend, intermittency, stationarity, complexity
- 15 forecasting methods: Statistical (10), Neural (5), Foundation (1), ML (2)
- Rolling-window backtesting with 10+ evaluation metrics (MAE, RMSE, MAPE, CRPS, Coverage...)
- Automated best method selection using weighted composite scoring
- Distribution fitting for safety stock and service level calculations
- REST API with 12 endpoints for programmatic access
- Interactive React dashboard with Vega-Lite visualizations
- 24-month forecast horizon with prediction intervals (50% and 90%)
- Date-range zoom slider for interactive exploration

Current Dataset

- 2,348 unique time series (item-site combinations)
- 42,934 historical data points
- Date range: January 2023 to January 2026
- Average 18.3 observations per series
- 5,058 forecasts generated across all methods and series
- 25 series with full backtest metrics (series with 36+ observations)

2. System Architecture

2.1 Technology Stack

Backend (Python 3.13)

Category	Library	Purpose
Data Processing	Pandas, NumPy, PyArrow	DataFrames, arrays, Parquet I/O
Database	DuckDB, psycopg2	PostgreSQL extraction via Duc..
Statistical	Nixtla StatsForecast	AutoARIMA, AutoETS, AutoTheta..
Neural	Nixtla NeuralForecast	NHITS, NBEATS, PatchTST, TFT...
Foundation	Google TimesFM	Pre-trained foundation model
ML	LightGBM, XGBoost	Gradient boosting models
API	FastAPI, Uvicorn	REST API server
Parallel	Dask	Distributed batch processing
Config	PyYAML	YAML configuration loading

Frontend (Node.js / React)

Category	Library	Version
Framework	React	18.2
Routing	React Router	v6.20
Visualization	Vega-Lite + react-vega	v5.16 / v7.6
HTTP Client	Axios	v1.6
Styling	Tailwind CSS	v3.4
Build Tool	Vite	v5.0

2.2 Pipeline Architecture

The system follows a sequential pipeline architecture with 6 main stages. Each stage reads from and writes to Parquet files, enabling any stage to be re-run independently. The pipeline is orchestrated via run_pipeline.py, which supports both full runs and selective step execution.

Pipeline flow:

1. ETL: PostgreSQL -> DuckDB -> Pandas -> Parquet
2. Characterization: Pattern detection per series
3. Forecasting: Generate 24-month forecasts with all eligible methods
4. Backtesting: Rolling-window cross-validation (3 windows)
5. Best Method: Weighted composite scoring to rank methods per series
6. Distribution Fitting: Parametric fits for MEIO service levels

2.3 Directory Structure

```
ForecastAI2026.01/files/
  api/
    main.py                    # FastAPI backend (12 endpoints)
  characterization/
    characterization.py        # Time series pattern detection
  config/
    config.yaml                # Central configuration
  data/
    time_series.parquet        # ETL output (42,934 rows)
  distribution/
    fitting.py                 # Distribution fitting for MEIO
  etl/
    etl.py                     # PostgreSQL extraction pipeline
  evaluation/
    metrics.py                 # Backtesting & evaluation metrics
  forecasting/
    statistical_models.py      # StatsForecast wrapper (10 models)
    neural_models.py           # NeuralForecast wrapper (5 models)
    foundation_models.py       # TimesFM wrapper
    ml_models.py               # LightGBM, XGBoost
  frontend/
    src/components/
      Dashboard.jsx            # Main dashboard view
      TimeSeriesViewer.jsx     # Series detail view
    vite.config.js             # Dev server + API proxy
  output/                      # Pipeline output Parquet files
  selection/
    best_method.py             # Method ranking & selection
  utils/
    orchestrator.py            # Dask parallel orchestration
  run_pipeline.py              # CLI entry point
  requirements.txt             # Python dependencies
```

3. Data Model

3.1 Source Database (Neon PostgreSQL)

The system extracts demand data from a Neon-hosted PostgreSQL database. The schema uses three tables in the "plan" schema: demand_actual (fact table), item (SKU dimension), and site (location dimension). The unique identifier for each time series is formed by concatenating item_id and site_id (e.g., "14787_1006").

Source Tables

Table	Key Columns	Description
plan.demand_a..	item_id, site_id, date..	Demand fact records
plan.item	id, name, xuid	Item/SKU dimension
plan.site	id, name, xuid	Site/location dimension

ETL Join Query

```
SELECT
  CONCAT(d.item_id, '_', d.site_id) AS unique_id,
  d.date,
  d.qty AS y
FROM plan.demand_actual d
LEFT JOIN plan.item i ON d.item_id = i.id
LEFT JOIN plan.site s ON d.site_id = s.id
WHERE d.date >= '2020-01-01'
ORDER BY unique_id, d.date;
```

3.2 Output Parquet Files

The pipeline produces 6 Parquet files, each serving a specific role:

data/time_series.parquet - Historical Demand Data

Column	Type	Description
unique_id	string	Item-site combination identifier (e.g., "14787_1006")
date	datetime	Monthly date of observation
y	float64	Demand quantity value

Statistics: 42,934 rows, 2,348 unique series, range 2023-01 to 2026-01.

output/time_series_characteristics.parquet - Series Profiles

Column	Type	Description
unique_id	string	Series identifier
n_observations	int	Number of historical data points
has_seasonality	bool	Seasonal pattern detected via ACF test
seasonal_strength	float	Autocorrelation strength (0-1)
has_trend	bool	Monotonic trend via Mann-Kendall test

trend_direction	string	up, down, or none
is_intermittent	bool	Intermittent demand (high zero ratio)
complexity_score	float	Composite complexity (0-1)
complexity_level	string	low (<0.35), medium, high (>0.65)
recommended_methods	list	Suggested forecasting methods

output/forecasts_all_methods.parquet - Forecast Results

Column	Type	Description
unique_id	string	Series identifier
method	string	Forecasting method name
point_forecast	list[float]	Mean forecast values (24 months)
quantiles	dict	Prediction quantiles (5th to 99th percentile)
hyperparameters	dict	Method-specific fitted parameters
training_time	float	Model fitting time in seconds

output/backtest_metrics.parquet - Accuracy Evaluation

Column	Type	Description
unique_id	string	Series identifier
method	string	Forecasting method
mae	float	Mean Absolute Error
rmse	float	Root Mean Squared Error
mape	float	Mean Absolute Percentage Error
bias	float	Forecast bias (positive = over-forecast)
coverage_90	float	90% prediction interval coverage
n_windows	int	Number of backtest rolling windows

output/best_method_per_series.parquet - Winner Selection

Column	Type	Description
unique_id	string	Series identifier
best_method	string	Winning forecasting method
best_score	float	Composite score (lower = better)
runner_up_method	string	Second-best method

output/forecasts_by_origin.parquet - Backtest Forecasts

Contains per-window forecasts for each rolling backtest origin. Used to power the "Forecast Evolution Over Time" animation in the frontend. Columns: unique_id, method, forecast_origin, horizon_step, point_forecast, actual_value.

4. Pipeline Stages

4.1 ETL (Extract-Transform-Load)

The ETL stage extracts demand data from a Neon PostgreSQL database using DuckDB's postgres_scanner extension, which provides high-performance parallel reads. The extracted data is transformed by parsing dates, casting values to numeric, creating unique identifiers, aggregating to monthly frequency, and filtering by minimum observation count (12 months). Output is saved as a Parquet file with Snappy compression.

Key Parameters

- Aggregation frequency: Monthly (M), using sum
- Minimum observations: 12 per series
- Date filter: >= 2020-01-01
- Output: data/time_series.parquet

4.2 Time Series Characterization

Each time series is analyzed along 5 dimensions to understand its patterns and determine which forecasting methods are most appropriate:

Dimension	Test	Output
Seasonality	ACF at config lags	has_seasonality, strength (0-1)
Trend	Mann-Kendall test	has_trend, direction (up/down)
Intermittency	Zero ratio + ADI	is_intermittent, zero_ratio, adi
Stationarity	ADF test	is_stationary, adf_pvalue
Complexity	Weighted composite	score (0-1), level (low/med/high)

Complexity Score Formula

```
Score = 0.20 * CV + 0.25 * Entropy + 0.15 * TurningPoints
      + 0.15 * NonStationarity + 0.10 * SeasonalStrength
      + 0.15 * Intermittency

Classification: Low (<0.35), Medium (0.35-0.65), High (>0.65)
```

Method Recommendation Logic

- Sparse data (<24 obs): SeasonalNaive, HistoricAverage, TimesFM
- Intermittent demand: CrostonOptimized, ADIDA, IMAPA, TimesFM
- Seasonal patterns: MSTL, AutoETS, NHITS, PatchTST, TimesFM, LightGBM
- High complexity: NHITS, NBEATS, TFT, PatchTST, AutoETS, LightGBM, XGBoost
- Standard (default): AutoETS, AutoARIMA, AutoTheta, NHITS, TimesFM, LightGBM

4.3 Forecasting

The system supports up to 15 forecasting methods across 4 families. For each series, methods are selected based on the characterization results. All methods generate 24-month ahead point forecasts along with prediction quantiles at 7 confidence levels (10th through 99th percentile) for use in probabilistic inventory optimization.

Statistical Models (Nixtla StatsForecast)

Method	Description
AutoARIMA	Auto-selected ARIMA with automatic differencing, order selection
AutoETS	Exponential Smoothing with automatic error/trend/seasonal selec..
AutoTheta	Theta method with automatic decomposition
AutoCES	Complex Exponential Smoothing
MSTL	Multiple Seasonal-Trend decomposition using LOESS
CrostonOptimized	Optimized Croston method for intermittent demand
ADIDA	Aggregate-Disaggregate Intermittent Demand Approach
IMAPA	Intermittent Multiple Aggregation Prediction Algorithm
HistoricAverage	Simple historical mean (baseline)
SeasonalNaive	Last seasonal cycle repeated (baseline)

Neural Models (Nixtla NeuralForecast)

Method	Description
NHITS	N-HiTS: Neural Hierarchical Interpolation for Time Series
NBEATS	Neural Basis Expansion Analysis for Time Series
PatchTST	Patch Time Series Transformer
TFT	Temporal Fusion Transformer
DeepAR	Probabilistic autoregressive RNN model

Other Models

- TimesFM (Google): Pre-trained 200M parameter foundation model (timesfm-1.0-200m)
- LightGBM: Gradient boosted decision trees with lag features
- XGBoost: Extreme gradient boosting with lag features

4.4 Backtesting & Evaluation

Forecast accuracy is evaluated using rolling-window cross-validation. The system uses 3 rolling windows with a step size of 1 month and a minimum training set of 12 observations. With a 24-month horizon, only series with 36+ observations qualify for backtesting (currently 57 series; 25 have enough methods for ranking).

Evaluation Metrics

Category	Metric	Interpretation
Point	MAE	Average absolute error (scale-dependent)
Point	RMSE	Penalizes large errors more than MAE
Point	MAPE	Percentage error (undefined if actual=0)
Point	sMAPE	Symmetric MAPE (handles zeros better)
Point	Bias	Systematic over/under-forecasting

Point	MASE	Scaled error vs naive baseline
Probabilistic	CRPS	Overall distributional accuracy
Probabilistic	Winkler Score	Prediction interval quality
Probabilistic	Coverage (90%)	Should be close to 90%
Info Criteria	AIC / BIC	Model parsimony assessment

4.5 Best Method Selection

For each series with backtest metrics, methods are ranked using a weighted composite score. The method with the lowest score is selected as the winner. The scoring formula is:

```
Composite Score = 0.40 * MAE_normalized
                + 0.20 * RMSE_normalized
                + 0.15 * |Bias|_normalized
                + 0.15 * (1 - Coverage_90)
                + 0.10 * MASE_normalized

Lower score = better method
```

The weights reflect a preference for point forecast accuracy (60% for MAE + RMSE), while still penalizing systematic bias and poor prediction interval calibration.

4.6 Distribution Fitting (MEIO)

For Multi-Echelon Inventory Optimization, parametric probability distributions are fitted to the forecast quantiles of the best method. Four distribution families are tested: Normal, Gamma, Negative Binomial, and Lognormal. The Kolmogorov-Smirnov test is used to assess goodness of fit, and service level quantiles at 90%, 95%, and 99% are calculated for safety stock computation.

5. API Reference

The FastAPI backend runs on port 8000 and serves all forecast data via REST endpoints. All responses are JSON. The API loads Parquet files into an in-memory cache on startup for fast response times.

GET /api/series

List all time series with filtering and pagination

Parameters:

skip (int), limit (int, max 50000), search (str), complexity (str), intermittent (bool)

Response:

Array of TimeSeriesInfo objects

GET /api/series/{id}/data

Get historical data for a specific series

Parameters:

unique_id (path)

Response:

Object with date[] and value[] arrays

GET /api/forecasts/{id}

Get all forecasts for a series

Parameters:

unique_id (path), methods (list, optional)

Response:

Object with forecasts[{method, point_forecast, quantiles}]

GET /api/metrics/{id}

Get backtest metrics for a series

Parameters:

unique_id (path)

Response:

Object with metrics[{method, mae, rmse, bias, mape, coverage_90}]

GET /api/analytics

Dashboard-level summary statistics

Parameters:

None

Response:

Totals, distributions, method counts

GET /api/best-methods

Get best method for all series

Parameters:

None

Response:

Array of {unique_id, best_method, best_score}

GET /api/series/{id}/best-method

Best method for one series

Parameters:

unique_id (path)

Response:

Object with best_method, best_score, runner_up

GET /api/forecasts/{id}/origins

List backtest origin dates

Parameters:

unique_id (path)

Response:

Object with origins[] date array

GET /api/forecasts/{id}/origins/{date}

Forecasts at a specific origin

Parameters:

unique_id, origin_date (path)

Response:

Object with forecasts[{method, point_forecast, actual}]

GET /api/series/{id}/forecast-evolution

Full evolution for animation

Parameters:

unique_id (path)

Response:

Object with evolution[{origin, forecasts}]

GET /api/series/{id}/vega-spec

Vega-Lite chart specification

Parameters:

unique_id (path)

Response:

Complete Vega-Lite v5 JSON spec

6. Frontend User Guide

The ForecastAI frontend is a single-page application built with React and Vega-Lite. It provides two main views: the Dashboard (overview of all series) and the Time Series Detail View (deep-dive into a single series). The frontend communicates with the API via a Vite development proxy on port 5173.

6.1 Dashboard

The Dashboard is the landing page and provides a high-level overview of all 2,348 time series. It features 6 summary cards, 2 interactive charts, search and filter controls, and a sortable paginated table of all series.

Summary Cards

- Total Series: Count of all time series in the dataset (2,348)
- Backtested: Number of series with backtest metrics (25 with 24-month horizon)
- Seasonal: Series with detected seasonality (143)
- Trending: Series with detected monotonic trend (415)
- Intermittent: Series with intermittent demand patterns (0 in current dataset)
- Avg Observations: Average number of monthly data points per series (18)

Charts

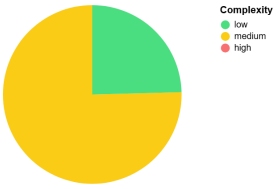
- Complexity Distribution (pie chart): Shows proportion of low vs. medium vs. high complexity series
- Best Method Distribution (bar chart): Shows which forecasting method wins most often across backtested series

ForecastAI Dashboard

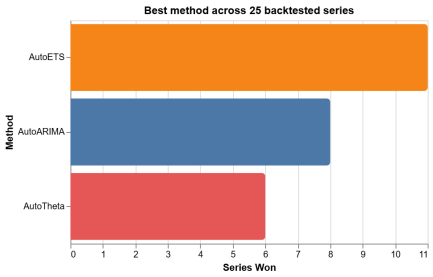
Forecasting Dashboard

Total Series	Backtested	Seasonal	Trending	Intermittent	Avg Observations
2,348	25	143	415	0	18

Complexity Distribution



Best Method Distribution



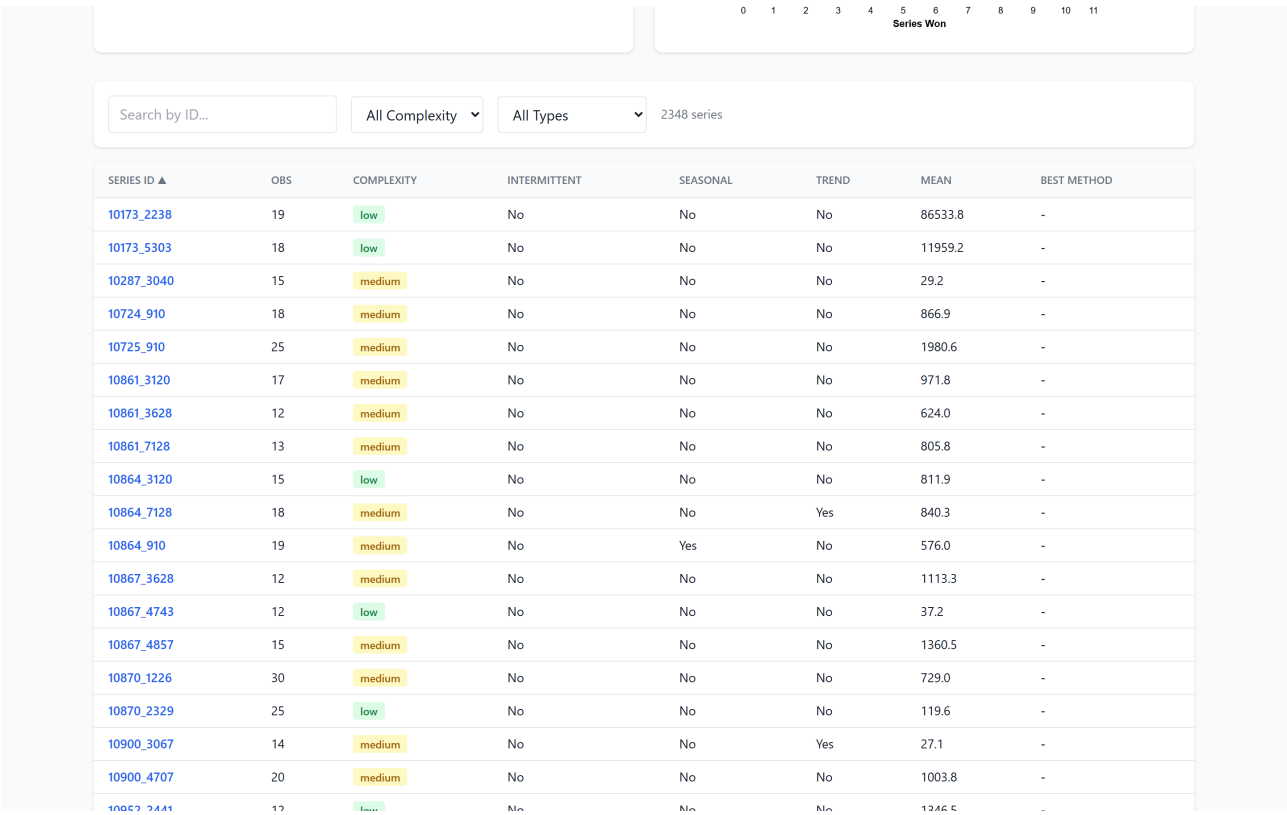
Search by ID... | All Complexity | All Types | 2348 series

SERIES ID ▲	OBS	COMPLEXITY	INTERMITTENT	SEASONAL	TREND	MEAN	BEST METHOD
10173_2238	19	low	No	No	No	86533.8	-
10173_5303	18	low	No	No	No	11959.2	-
10287_3040	15	medium	No	No	No	29.2	-
10724_910	18	medium	No	No	No	866.9	-
10725_910	25	medium	No	No	No	1980.6	-
10861_3120	17	medium	No	No	No	971.8	-
10861_3628	12	medium	No	No	No	624.0	-
10861_7128	13	medium	No	No	No	805.8	-
10864_3120	15	low	No	No	No	811.9	-
10864_7128	18	medium	No	No	Yes	840.3	-
10864_910	19	medium	No	Yes	No	576.0	-
10867_3628	12	medium	No	No	No	1113.3	-
10867_4743	12	low	No	No	No	37.2	-
10867_4857	15	medium	No	No	No	1360.5	-
10870_1226	30	medium	No	No	No	729.0	-
10870_2329	25	low	No	No	No	119.6	-
10900_3067	14	medium	No	No	Yes	27.1	-
10900_4707	20	medium	No	No	No	1003.8	-
10952_2441	12	low	No	No	No	1346.5	-
10952_3127	16	medium	No	No	No	97.8	-
10984_2212	15	low	No	No	No	1145.9	-
11035_910	12	medium	No	No	No	454.6	-
11039_14870	19	medium	No	Yes	No	3430.7	-
11066_7480	12	medium	No	No	No	5333.8	-
11074_7480	12	medium	No	No	No	10458.3	-
11121_1359	12	low	No	No	No	892.2	-
11122_1359	12	medium	No	No	No	993.8	-
11419_1385	17	medium	No	No	No	266.5	-
11419_2375	18	medium	No	Yes	No	187.8	-
11419_3054	15	medium	No	No	No	134.1	-
11425_1044	13	medium	No	No	Yes	38.7	-
11425_1385	13	medium	No	No	No	217.4	-
11533_2761	12	medium	No	No	No	12387.5	-
11721_1101	28	medium	No	No	Yes	184.2	-
11721_7234	29	low	No	No	No	545.4	-
11950_6094	13	medium	No	No	No	180.6	-
11950_6517	16	medium	No	No	No	142.1	-
11987_2441	16	medium	No	No	No	77.8	-
12123_2761	16	medium	No	No	Yes	18307.5	-
12270_7889	12	low	No	No	No	56.8	-
13937_7889	14	medium	No	No	Yes	503.6	-
13937_910	15	low	No	No	No	1235.0	-

Figure 1: Dashboard - Summary cards, charts, filters, and series table

Series Table

The table lists all series with columns: Series ID, Observations, Complexity, Intermittent, Seasonal, Trend, Mean, and Best Method. Clicking any row navigates to the detail view. The table supports search by ID, filtering by complexity level and intermittency, and pagination (50 rows per page).



SERIES ID ▲	OBS	COMPLEXITY	INTERMITTENT	SEASONAL	TREND	MEAN	BEST METHOD
10173_2238	19	low	No	No	No	86533.8	-
10173_5303	18	low	No	No	No	11959.2	-
10287_3040	15	medium	No	No	No	29.2	-
10724_910	18	medium	No	No	No	866.9	-
10725_910	25	medium	No	No	No	1980.6	-
10861_3120	17	medium	No	No	No	971.8	-
10861_3628	12	medium	No	No	No	624.0	-
10861_7128	13	medium	No	No	No	805.8	-
10864_3120	15	low	No	No	No	811.9	-
10864_7128	18	medium	No	No	Yes	840.3	-
10864_910	19	medium	No	Yes	No	576.0	-
10867_3628	12	medium	No	No	No	1113.3	-
10867_4743	12	low	No	No	No	37.2	-
10867_4857	15	medium	No	No	No	1360.5	-
10870_1226	30	medium	No	No	No	729.0	-
10870_2329	25	low	No	No	No	119.6	-
10900_3067	14	medium	No	No	Yes	27.1	-
10900_4707	20	medium	No	No	No	1003.8	-

Figure 2: Dashboard - Series table with filters and pagination

6.2 Time Series Detail View

The detail view is the core analytical tool. It shows everything about a single time series: its characteristics, all forecast methods with their predictions, accuracy metrics from backtesting, and the evolution of forecasts over time.

Header & Characteristics

The header shows the series ID and characteristic badges: observation count, continuous vs. intermittent, seasonal vs. non-seasonal, trending vs. stationary, complexity level, and the winning forecast method (if backtest data is available).

Method Toggles

Color-coded toggle buttons let you show/hide individual forecasting methods on all charts. Each method has a unique color. The best method is marked with a star icon.

Main Forecast Chart

The main chart overlays historical data (solid black line) with forecast lines (dashed, color-coded per method) and prediction intervals (shaded bands for 50% and 90% confidence). The chart title indicates the horizon length (24-month). A Vega-Lite tooltip shows exact values on hover.

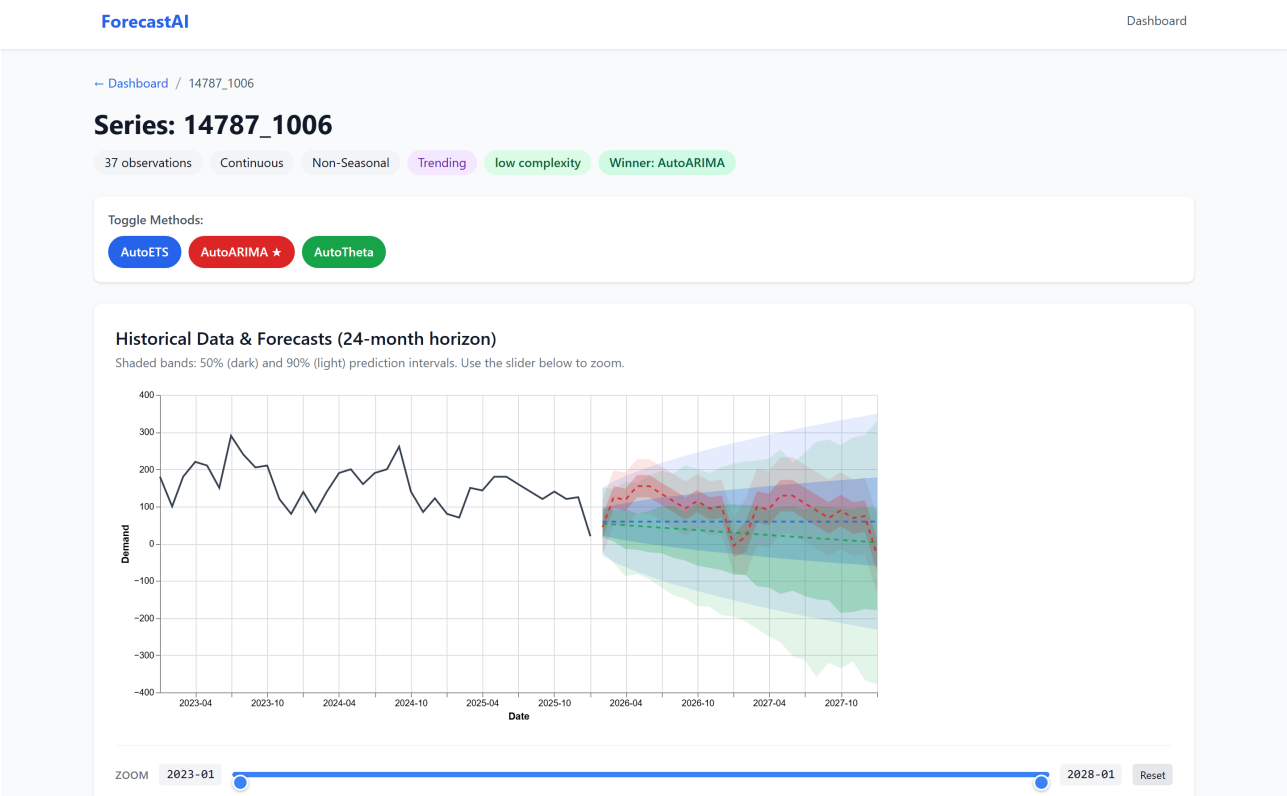


Figure 3: Series detail - Header, method toggles, and main forecast chart with 24-month horizon

Backtest Metrics

Two panels show accuracy results from backtesting: a horizontal bar chart comparing MAE across methods (green border highlights the winner), and a detailed table with MAE, RMSE, Bias, MAPE, 90% Coverage, and number of backtest windows. For series without enough history for backtesting, a note explains the minimum observation requirement.

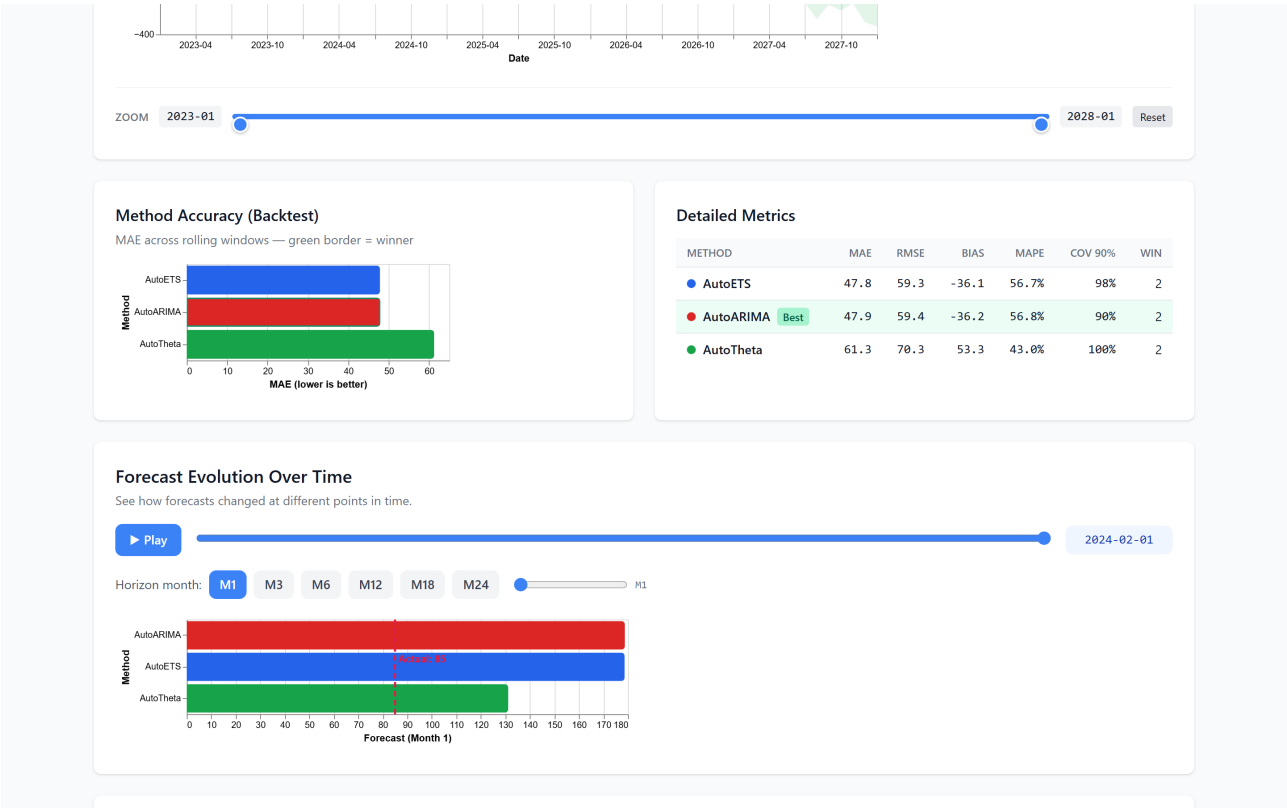


Figure 4: Backtest metrics - MAE bar chart and detailed metrics table

Forecast Evolution & Racing Bars

The "Forecast Evolution Over Time" section shows how forecasts changed at different backtest origins. A Play/Pause button animates through origins. A slider allows manual origin selection. The "racing bars" chart compares forecast values across methods for a selected horizon month, with a red dashed line indicating the actual value when available.

The horizon month selector provides quick-access buttons (M1, M3, M6, M12, M18, M24) plus a fine-grained slider for any month in between.

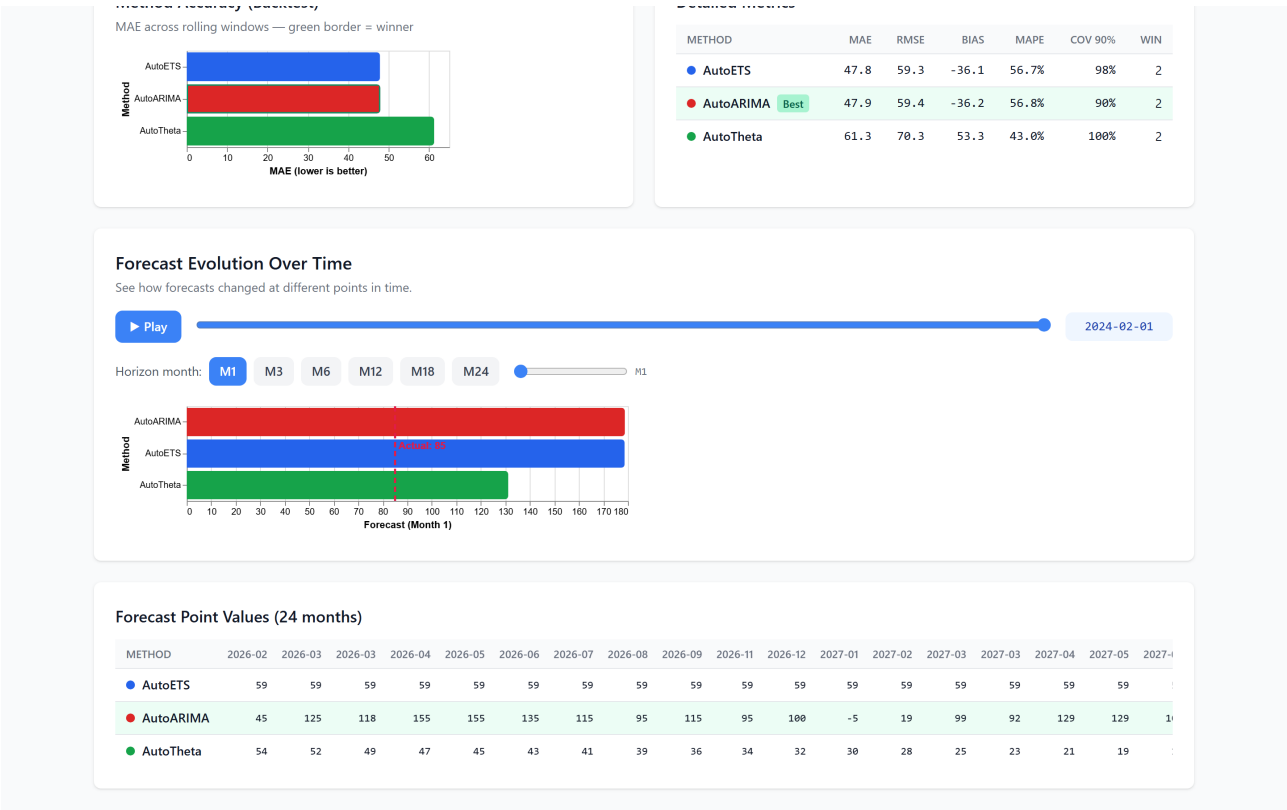


Figure 5: Forecast evolution - Origin slider, racing bars, and horizon selector

Forecast Values Table

At the bottom, a horizontally scrollable table shows the exact point forecast values for all 24 months across all methods. The method column is sticky (stays visible while scrolling). The best method row is highlighted in green.

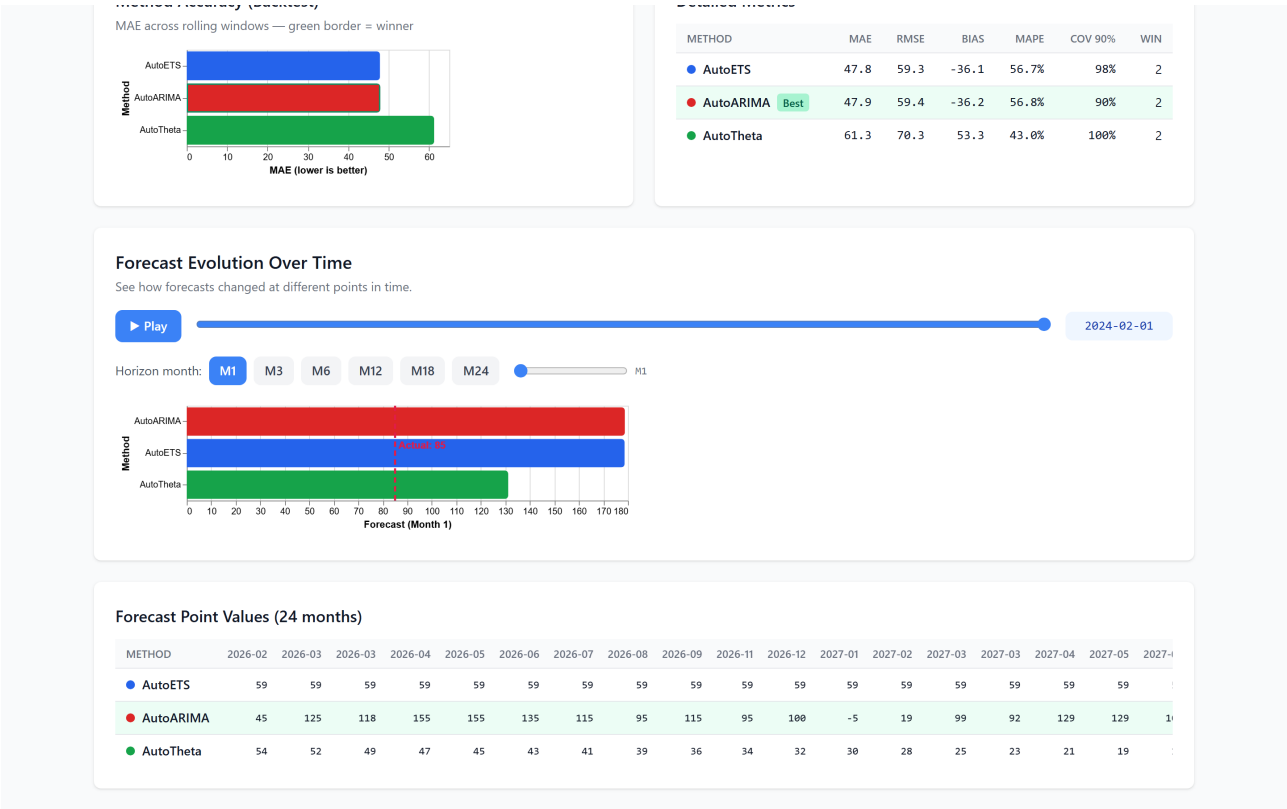


Figure 6: Forecast point values table (24 months, all methods)

6.3 Interactive Features

Date Range Zoom Slider

Below the main chart, a dual-handle slider allows zooming into any date range. The left handle sets the start date and the right handle sets the end date. A blue highlighted bar shows the selected range. The chart updates in real-time as you drag. A "Reset" button returns to the full date range.

Method Toggles

Click any method button to show/hide it across all charts. Active methods are shown with their color background; inactive ones are grayed out. This helps compare specific method subsets.

Forecast Evolution Animation

Click "Play" to animate through all backtest origins, watching how forecasts evolved. The animation advances every 800ms. Click "Stop" to pause at any point.

7. Configuration Reference

All system parameters are controlled via config/config.yaml. Key sections include:

Data Source

```
data_source:
  type: "postgres"
  postgres:
    host: "ep-gentle-rain-a5ba1hxu.us-east-2.aws.neon.tech"
    port: 5432
    database: "scenario"
    schema: "plan"
    sslmode: "require"
```

ETL Settings

```
etl:
  output_path: "./data/time_series.parquet"
  query:
    date_column: "date"
    value_column: "qty"
    min_observations: 12
  aggregation:
    frequency: "M"      # Monthly
    method: "sum"
```

Forecasting

```
forecasting:
  horizon: 24              # 24-month forecast
  frequency: "M"
  confidence_levels: [10, 25, 50, 75, 90, 95, 99]
  backtesting:
    n_windows: 3           # Rolling windows
    step_size: 1
    min_train_size: 12
```

Best Method Selection Weights

```
best_method:
  weights:
    mae: 0.40
    rmse: 0.20
    bias: 0.15
    coverage_90: 0.15
    mase: 0.10
```

MEIO Distribution Fitting

```
meio:
  distributions: [normal, gamma, negative_binomial, lognormal]
  fitting_method: "quantile_matching"
  service_levels: [0.90, 0.95, 0.99]
```

Parallel Processing

```
parallel:
  backend: "dask"
  dask:
    scheduler: "processes"
    n_workers: null        # Auto-detect CPU cores
  batch_size: 100
```

8. Installation & Running

Prerequisites

- Python 3.11+ (tested with 3.13)
- Node.js 18+ with npm
- Network access to Neon PostgreSQL (for ETL step)

Backend Setup

```
cd files/
pip install -r requirements.txt
```

Frontend Setup

```
cd files/frontend/
npm install
```

Running the Full Pipeline

```
# Run all stages (ETL through distribution fitting)
python run_pipeline.py

# Skip ETL (use existing data/time_series.parquet)
python run_pipeline.py --skip-etl

# Run only a specific stage
python run_pipeline.py --only etl
python run_pipeline.py --only characterize
python run_pipeline.py --only forecast
python run_pipeline.py --only backtest
python run_pipeline.py --only best-method
python run_pipeline.py --only distributions

# Discover database schema
python run_pipeline.py --discover-schema
```

Starting the API Server

```
# From the files/ directory
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload

# Or directly:
python api/main.py
```

The API loads all Parquet files into memory on startup and serves at http://localhost:8000.

Starting the Frontend

```
cd files/frontend/
npm run dev

# For production build:
npm run build
npm run preview
```

The Vite dev server starts on http://localhost:5173 and automatically proxies /api/* requests to the backend on port 8000. Open http://localhost:5173 in a browser to access the dashboard.

Quick Start (Both Servers)

```
# Terminal 1: Start API
cd files/ && uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload

# Terminal 2: Start Frontend
cd files/frontend/ && npm run dev

# Open http://localhost:5173 in your browser
```

9. Troubleshooting

Dashboard shows 0 series in the table

This can happen if the API limit parameter is too restrictive. The frontend requests limit=50000. Verify the API is running and responding: curl http://localhost:8000/api/series?limit=5

Charts appear empty in the detail view

Ensure the API server was restarted after regenerating Parquet files. The API caches data in memory at startup. If the Parquet files changed, restart the API with: uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload

Port already in use

If port 8000 or 5173 is occupied:
Windows: netstat -ano | findstr :8000 then taskkill /F /PID <pid>
Linux/Mac: lsof -i :8000 then kill <pid>

Few series have backtest metrics

With a 24-month forecast horizon and min_train_size=12, a series needs at least 36 monthly observations. Only 57 of 2,348 series meet this threshold. To increase backtesting coverage, reduce the horizon in config.yaml (e.g., horizon: 6) or reduce min_train_size. Note that this is a data limitation, not a system bug.

Vite dev server keeps dying

The Vite dev server may crash if system memory is low. Try restarting with: cd files/frontend && npx vite --host. For production use, build the frontend with npm run build and serve the static files.

Database connection failures

Verify the Neon PostgreSQL credentials in config/config.yaml. Neon databases may auto-suspend after inactivity. The first connection may take a few seconds as the compute endpoint wakes up. Ensure sslmode: "require" is set for Neon connections.